CMIP6 Climate Data Analysis

GEOG-434 Final Project

Analysis of Surface Temperature, Precipitation, and Soil Moisture under Different Climate Scenarios

Author: Mario Jinesta

Date: December 2025

This Jupyter Notebook analyzes climate model data from the Coupled Model Intercomparison Project Phase 6 (CMIP6). We examine how three key climate variables change under different future scenarios:

- **tas**: Surface Air Temperature (measured in Kelvin, converted to °C)
- **pr**: Precipitation (measured in $kg/m^2/s$)
- **mrsos**: Surface Soil Moisture (measured in kg/m^2)

We use the NASA-GISS GISS-E2-1-G model with 4 different ensemble members and compare:

- **Historical period** (1850-2014): What actually happened based on observed forcings
- **SSP1-2.6**: Low emissions scenario representing sustainable development
- **SSP3-7.0**: High emissions scenario representing regional rivalry and continued fossil fuel use

---

Part 1: Import Libraries

Libraries are pre-written code packages that provide tools for specific tasks. We import several libraries:

- **numpy**: For numerical operations and array mathematics
- **xarray**: For working with labeled multi-dimensional arrays (perfect for climate data with time, latitude, longitude)
- **matplotlib**: For creating visualizations and plots
- **cartopy**: For creating maps with geographic projections
- **scipy**: For statistical analysis including trend calculations
- **glob**: For finding files that match specific patterns

In [1]:
```
# Import required libraries
import numpy as np
import xarray as xr
import matplotlib.pyplot as plt
import cartopy.crs as ccrs
import cartopy.feature as cfeature
from scipy import stats
import glob
import os
import warnings

# Suppress warnings to keep output clean
warnings.filterwarnings('ignore')

print("All libraries imported successfully!")
```

```
All libraries imported successfully!
```

Part 2: Set Data Path

We specify the location of our downloaded CMIP6 NetCDF files. NetCDF (Network Common Data Form) is a standard file format for storing scientific array data, particularly climate and atmospheric data. Each file contains gridded data with dimensions for time, latitude, and longitude.

In [2]:
```
# Set the path to your data folder
DATA_PATH = r"C:\Users\mario\python_scripts\cmip6_data"

# Verify the path exists
if os.path.exists(DATA_PATH):
    print(f"Data directory found: {DATA_PATH}")
    print(f"Number of files: {len(os.listdir(DATA_PATH))}")
else:
    print(f"ERROR: Directory not found: {DATA_PATH}")
    print("Please update the DATA_PATH variable above.")
```

```
Data directory found: C:\Users\mario\python_scripts\cmip6_data
Number of files: 197
```

Part 3: Load Climate Data

We create functions to load and organize our climate data. The files are split into multiple time chunks, so we need to find all files for each variable, scenario, and ensemble member, then combine them into continuous datasets.

The ensemble members (r1i1p1f2, r2i1p1f2, etc.) represent different model runs with slightly different initial conditions. This helps us understand the range of possible climate outcomes.

In [3]:
```
def load_variable_data(data_path, variable, scenario, ensemble_members):
    """
    Load all NetCDF files for a specific variable, scenario, and ensemble members.
    Converts to annual means for faster processing.
    """
    # Determine the correct table ID based on variable
    if variable == 'mrsos':
        table_id = 'Lmon'
    else:
        table_id = 'Amon'

    datasets = {}

    for member in ensemble_members:
        pattern = os.path.join(data_path, f"{variable}_{table_id}_GISS-E2-1-G_{scen
        files = sorted(glob.glob(pattern))

        if len(files) > 0:
            # Open and combine files, then convert to annual means
            ds = xr.open_mfdataset(files, combine='by_coords', engine='netcdf4')
            # Resample to annual means for faster processing
            ds_annual = ds.resample(time='YE').mean()
            datasets[member] = ds_annual
            print(f"  Loaded {variable} {scenario} {member}: {len(files)} files ->
        else:
            print(f"  WARNING: No files found for {variable} {scenario} {member}")

    return datasets

print("Data loading function defined.")
```

```
Data loading function defined.
```

In [4]:
```
# Define ensemble members, variables, and scenarios
ensemble_members = ['r1i1p1f2', 'r2i1p1f2', 'r3i1p1f2', 'r4i1p1f2']
variables = ['tas', 'pr', 'mrsos']
scenarios = ['historical', 'ssp126', 'ssp370']

# Load all data
all_data = {}

print("Loading climate data (converting to annual means for efficiency)...")
print("=" * 60)

for scenario in scenarios:
    print(f"\n{scenario.upper()}:")
    all_data[scenario] = {}
    for variable in variables:
        all_data[scenario][variable] = load_variable_data(
            DATA_PATH, variable, scenario, ensemble_members
        )

print("\n" + "=" * 60)
print("Data loading complete!")
```

```
Loading climate data (converting to annual means for efficiency)...
============================================================

HISTORICAL:
  Loaded tas historical r1i1p1f2: 4 files -> annual means
  Loaded tas historical r2i1p1f2: 4 files -> annual means
  Loaded tas historical r3i1p1f2: 4 files -> annual means
  Loaded tas historical r4i1p1f2: 4 files -> annual means
  Loaded pr historical r1i1p1f2: 4 files -> annual means
  Loaded pr historical r2i1p1f2: 4 files -> annual means
  Loaded pr historical r3i1p1f2: 4 files -> annual means
  Loaded pr historical r4i1p1f2: 4 files -> annual means
  Loaded mrsos historical r1i1p1f2: 4 files -> annual means
  Loaded mrsos historical r2i1p1f2: 4 files -> annual means
  Loaded mrsos historical r3i1p1f2: 4 files -> annual means
  Loaded mrsos historical r4i1p1f2: 4 files -> annual means

SSP126:
  Loaded tas ssp126 r1i1p1f2: 10 files -> annual means
  Loaded tas ssp126 r2i1p1f2: 10 files -> annual means
  Loaded tas ssp126 r3i1p1f2: 10 files -> annual means
  Loaded tas ssp126 r4i1p1f2: 10 files -> annual means
  Loaded pr ssp126 r1i1p1f2: 10 files -> annual means
  Loaded pr ssp126 r2i1p1f2: 10 files -> annual means
  Loaded pr ssp126 r3i1p1f2: 10 files -> annual means
  Loaded pr ssp126 r4i1p1f2: 10 files -> annual means
  Loaded mrsos ssp126 r1i1p1f2: 10 files -> annual means
  Loaded mrsos ssp126 r2i1p1f2: 10 files -> annual means
  Loaded mrsos ssp126 r3i1p1f2: 10 files -> annual means
  Loaded mrsos ssp126 r4i1p1f2: 10 files -> annual means

SSP370:
  Loaded tas ssp370 r1i1p1f2: 2 files -> annual means
  Loaded tas ssp370 r2i1p1f2: 2 files -> annual means
  Loaded tas ssp370 r3i1p1f2: 2 files -> annual means
  Loaded tas ssp370 r4i1p1f2: 2 files -> annual means
  Loaded pr ssp370 r1i1p1f2: 2 files -> annual means
  Loaded pr ssp370 r2i1p1f2: 2 files -> annual means
  Loaded pr ssp370 r3i1p1f2: 2 files -> annual means
  Loaded pr ssp370 r4i1p1f2: 2 files -> annual means
  Loaded mrsos ssp370 r1i1p1f2: 2 files -> annual means
  Loaded mrsos ssp370 r2i1p1f2: 2 files -> annual means
  Loaded mrsos ssp370 r3i1p1f2: 2 files -> annual means
  Loaded mrsos ssp370 r4i1p1f2: 2 files -> annual means

============================================================
Data loading complete!
```

Part 4: Examine Data Structure

Let's examine what's inside our datasets. Each NetCDF file contains:

- **Dimensions**: time, latitude (lat), longitude (lon)

- **Coordinates**: The actual values for each dimension
- **Data Variables**: The climate variable values at each grid point and time step
- **Attributes**: Metadata including units, variable descriptions, and model information

In [5]:
```
# Examine one example dataset
example_ds = all_data['historical']['tas']['r1i1p1f2']

print("Example Dataset Structure (Historical Temperature, Member 1):")
print("=" * 70)
print(f"Time range: {str(example_ds.time.values[0])[:4]} to {str(example_ds.time.va
print(f"Number of years: {len(example_ds.time)}")
print(f"Grid resolution: {len(example_ds.lat)} x {len(example_ds.lon)} points")
print(f"Latitude range: {float(example_ds.lat.min()):.1f}° to {float(example_ds.lat
print(f"Longitude range: {float(example_ds.lon.min()):.1f}° to {float(example_ds.lo
```

```
Example Dataset Structure (Historical Temperature, Member 1):
======================================================================
Time range: 1850 to 2014
Number of years: 165
Grid resolution: 90 x 144 points
Latitude range: -89.0° to 89.0°
Longitude range: 1.2° to 358.8°
```

Part 5: Combine Historical and Future Scenarios

For trend analysis, we need continuous time series from 1850 to 2100. We combine:

- Historical data (1850-2014): Based on observed greenhouse gas concentrations and other forcings
- Future scenario data (2015-2100): Based on projected emissions pathways

This creates seamless datasets spanning the entire analysis period for each future scenario.

In [6]:
```
def combine_historical_future(hist_data, future_data):
    """Combine historical and future scenario data into continuous time series."""
    combined = {}
    for member in hist_data.keys():
        if member in future_data:
            combined[member] = xr.concat([hist_data[member], future_data[member]],
    return combined

# Combine data for both scenarios
print("Combining historical and future scenario data...")
print("=" * 60)

combined_data = {'ssp126': {}, 'ssp370': {}}

for variable in variables:
    print(f"{variable}:")
    combined_data['ssp126'][variable] = combine_historical_future(
        all_data['historical'][variable], all_data['ssp126'][variable])
```

```
    print(f"  SSP1-2.6: {len(combined_data['ssp126'][variable])} members")

    combined_data['ssp370'][variable] = combine_historical_future(
        all_data['historical'][variable], all_data['ssp370'][variable])
    print(f"  SSP3-7.0: {len(combined_data['ssp370'][variable])} members")

print("\nData combination complete!")
```

```
Combining historical and future scenario data...
============================================================
tas:
  SSP1-2.6: 4 members
  SSP3-7.0: 4 members
pr:
  SSP1-2.6: 4 members
  SSP3-7.0: 4 members
mrsos:
  SSP1-2.6: 4 members
  SSP3-7.0: 4 members

Data combination complete!
```

Part 6: Calculate Anomalies

An **anomaly** is the difference between a value and a reference period average. We use 1850-2015 as our reference (baseline) period.

$$\text{Anomaly} = \text{Value} - \text{Reference Period Mean}$$

Calculating anomalies helps us:

1. Remove the baseline climate state
2. Identify long-term changes relative to a baseline
3. Compare changes across different regions with different baseline climates

In [7]:
```
def calculate_anomalies(ds, variable, ref_start='1850', ref_end='2015'):
    """Calculate anomalies relative to a reference period."""
    var_data = ds[variable]
    ref_mean = var_data.sel(time=slice(ref_start, ref_end)).mean(dim='time')
    anomalies = var_data - ref_mean
    ds_anom = ds.copy()
    ds_anom[variable] = anomalies
    return ds_anom

# Calculate anomalies
print("Calculating anomalies (reference period: 1850-2015)...")
print("=" * 60)

anomaly_data = {'ssp126': {}, 'ssp370': {}}

for scenario in ['ssp126', 'ssp370']:
    print(f"\n{scenario.upper()}:")
```

```
    for variable in variables:
        anomaly_data[scenario][variable] = {}
        for member, ds in combined_data[scenario][variable].items():
            anomaly_data[scenario][variable][member] = calculate_anomalies(ds, vari
        print(f"  {variable}: {len(anomaly_data[scenario][variable])} members proce

print("\nAnomaly calculations complete!")
```

```
Calculating anomalies (reference period: 1850-2015)...
============================================================

SSP126:
  tas: 4 members processed
  pr: 4 members processed
  mrsos: 4 members processed

SSP370:
  tas: 4 members processed
  pr: 4 members processed
  mrsos: 4 members processed

Anomaly calculations complete!
```

Part 7: Calculate Linear Trends

A **trend** shows how much a variable changes over time. We use **linear regression** to calculate the rate of change:

$$y = mx + b$$

Where:

- y = climate variable value
- x = time
- m = **trend (slope)** - the rate of change we want to calculate
- b = intercept

We also calculate the **p-value** to determine statistical significance. A p-value < 0.05 indicates the trend is statistically significant at the 95% confidence level.

In [8]:
```
def calculate_trend_fast(ds, variable):
    """
    Calculate linear trend using vectorized operations for speed.
    Returns trend per decade and p-values.
    """
    var_data = ds[variable].values  # Convert to numpy for speed
    ntime, nlat, nlon = var_data.shape

    # Time as years
    x = np.arange(ntime)
```

```
    # Reshape for vectorized calculation
    y = var_data.reshape(ntime, -1)  # Shape: (time, lat*lon)

    # Calculate trends using polyfit (much faster than loop)
    # Handle NaN values by masking
    trends = np.zeros(y.shape[1])
    pvalues = np.ones(y.shape[1])

    for i in range(y.shape[1]):
        yi = y[:, i]
        mask = ~np.isnan(yi)
        if np.sum(mask) > 10:
            slope, intercept, r_value, p_value, std_err = stats.linregress(x[mask],
            trends[i] = slope * 10  # Per decade
            pvalues[i] = p_value
        else:
            trends[i] = np.nan
            pvalues[i] = 1.0

    # Reshape back to grid
    trends = trends.reshape(nlat, nlon)
    pvalues = pvalues.reshape(nlat, nlon)

    return trends, pvalues, ds.lat.values, ds.lon.values

print("Fast trend calculation function defined.")
```

```
Fast trend calculation function defined.
```

In [9]:

```
# Calculate trends for all datasets
print("Calculating trends (this may take a few minutes)...")
print("=" * 60)

trend_data = {'ssp126': {}, 'ssp370': {}}

for scenario in ['ssp126', 'ssp370']:
    print(f"\n{scenario.upper()}:")
    for variable in variables:
        print(f"  {variable}: ", end="", flush=True)
        trend_data[scenario][variable] = {'trends': [], 'pvalues': [], 'lat': None,

        for idx, (member, ds) in enumerate(anomaly_data[scenario][variable].items()
            print(f"m{idx+1} ", end="", flush=True)
            trend, pval, lat, lon = calculate_trend_fast(ds, variable)
            trend_data[scenario][variable]['trends'].append(trend)
            trend_data[scenario][variable]['pvalues'].append(pval)
            trend_data[scenario][variable]['lat'] = lat
            trend_data[scenario][variable]['lon'] = lon
        print("✓")

print("\n" + "=" * 60)
print("Trend calculations complete!")
```

```
Calculating trends (this may take a few minutes)...
============================================================

SSP126:
m1 m2 m3 m4 ✓
m1 m2 m3 m4 ✓
m1 m2 m3 m4 ✓

SSP370:
m1 m2 m3 m4 ✓
m1 m2 m3 m4 ✓
m1 m2 m3 m4 ✓

============================================================
Trend calculations complete!
```

Part 8: Calculate Multi-Ensemble Mean

The **multi-ensemble mean** averages results across all ensemble members. This approach:

1. Reduces the impact of internal climate variability
2. Provides the most likely outcome
3. Allows us to assess where trends are robust across different initial conditions

We determine **statistical significance** by checking if the majority of ensemble members show significant trends at each grid point.

In [10]:
```
def calculate_ensemble_statistics(trend_list, pvalue_list, alpha=0.05):
    """Calculate ensemble mean trend and significance."""
    trend_stack = np.stack(trend_list, axis=0)
    ensemble_mean = np.nanmean(trend_stack, axis=0)

    pval_stack = np.stack(pvalue_list, axis=0)
    sig_count = np.sum(pval_stack < alpha, axis=0)
    significance = sig_count > (len(pvalue_list) / 2)

    return ensemble_mean, significance

# Calculate ensemble statistics
print("Calculating multi-ensemble mean trends and significance...")
print("=" * 60)

ensemble_stats = {'ssp126': {}, 'ssp370': {}}

for scenario in ['ssp126', 'ssp370']:
    print(f"\n{scenario.upper()}:")
    for variable in variables:
        mean_trend, significance = calculate_ensemble_statistics(
            trend_data[scenario][variable]['trends'],
            trend_data[scenario][variable]['pvalues']
        )
```

```
        ensemble_stats[scenario][variable] = {
            'mean_trend': mean_trend,
            'significance': significance,
            'lat': trend_data[scenario][variable]['lat'],
            'lon': trend_data[scenario][variable]['lon']
        }

        pct_sig = np.sum(significance) / significance.size * 100
        print(f"  {variable}: {pct_sig:.1f}% significant")

print("\nEnsemble statistics complete!")
```

```
Calculating multi-ensemble mean trends and significance...
============================================================

SSP126:
  tas: 96.3% significant
  pr: 61.6% significant
  mrsos: 19.7% significant

SSP370:
  tas: 99.4% significant
  pr: 69.8% significant
  mrsos: 19.5% significant

Ensemble statistics complete!
```

Part 9: Create Global Trend Maps

Now we visualize our results with global maps showing:

1. **Spatial pattern of trends**: Color indicates the direction and magnitude of change
2. **Statistical significance**: Stippling marks regions where trends are significant across ensemble members

We use the Robinson projection, which provides a good balance between preserving shape and area for global maps.

In [11]:
```
def plot_trend_map(mean_trend, significance, lat, lon, variable, scenario):
    """
    Create a global map showing trends with significance stippling.
    """
    # Variable-specific parameters
    if variable == 'tas':
        cmap = 'RdBu_r'
        label = 'Temperature Trend (°C/decade)'
        vmax = 1.5
        var_name = 'Surface Air Temperature'
    elif variable == 'pr':
        cmap = 'BrBG'
        label = 'Precipitation Trend (mm/day/decade)'
```

```
        mean_trend = mean_trend * 86400  # Convert kg/m²/s to mm/day
        vmax = 0.15
        var_name = 'Precipitation'
    else:  # mrsos
        cmap = 'BrBG'
        label = 'Soil Moisture Trend (kg/m²/decade)'
        vmax = 3.0
        var_name = 'Surface Soil Moisture'

    scenario_labels = {'ssp126': 'SSP1-2.6 (Low Emissions)', 'ssp370': 'SSP3-7.0 (H

    # Create figure
    fig = plt.figure(figsize=(14, 8))
    ax = plt.axes(projection=ccrs.Robinson())
    ax.coastlines(linewidth=0.5)
    ax.add_feature(cfeature.BORDERS, linestyle=':', linewidth=0.3)
    ax.set_global()

    # Create meshgrid
    lon_2d, lat_2d = np.meshgrid(lon, lat)

    # Plot trends
    im = ax.pcolormesh(lon_2d, lat_2d, mean_trend, transform=ccrs.PlateCarree(),
                       cmap=cmap, vmin=-vmax, vmax=vmax)

    # Add stippling for significance
    sig_lons, sig_lats = lon_2d[significance], lat_2d[significance]
    ax.scatter(sig_lons[::3], sig_lats[::3], c='black', s=0.5, alpha=0.5,
               transform=ccrs.PlateCarree())

    # Colorbar and title
    cbar = plt.colorbar(im, ax=ax, orientation='horizontal', pad=0.05, shrink=0.8)
    cbar.set_label(label, fontsize=12)
    ax.set_title(f'{var_name} Trend\n{scenario_labels[scenario]} (1850-2100)', font

    plt.tight_layout()
    return fig

print("Map plotting function defined.")
```

```
Map plotting function defined.
```

Temperature (TAS) Trend Maps

In [12]:

```
# TAS SSP126
fig = plot_trend_map(
    ensemble_stats['ssp126']['tas']['mean_trend'],
    ensemble_stats['ssp126']['tas']['significance'],
    ensemble_stats['ssp126']['tas']['lat'],
    ensemble_stats['ssp126']['tas']['lon'],
    'tas', 'ssp126')
plt.show()
```

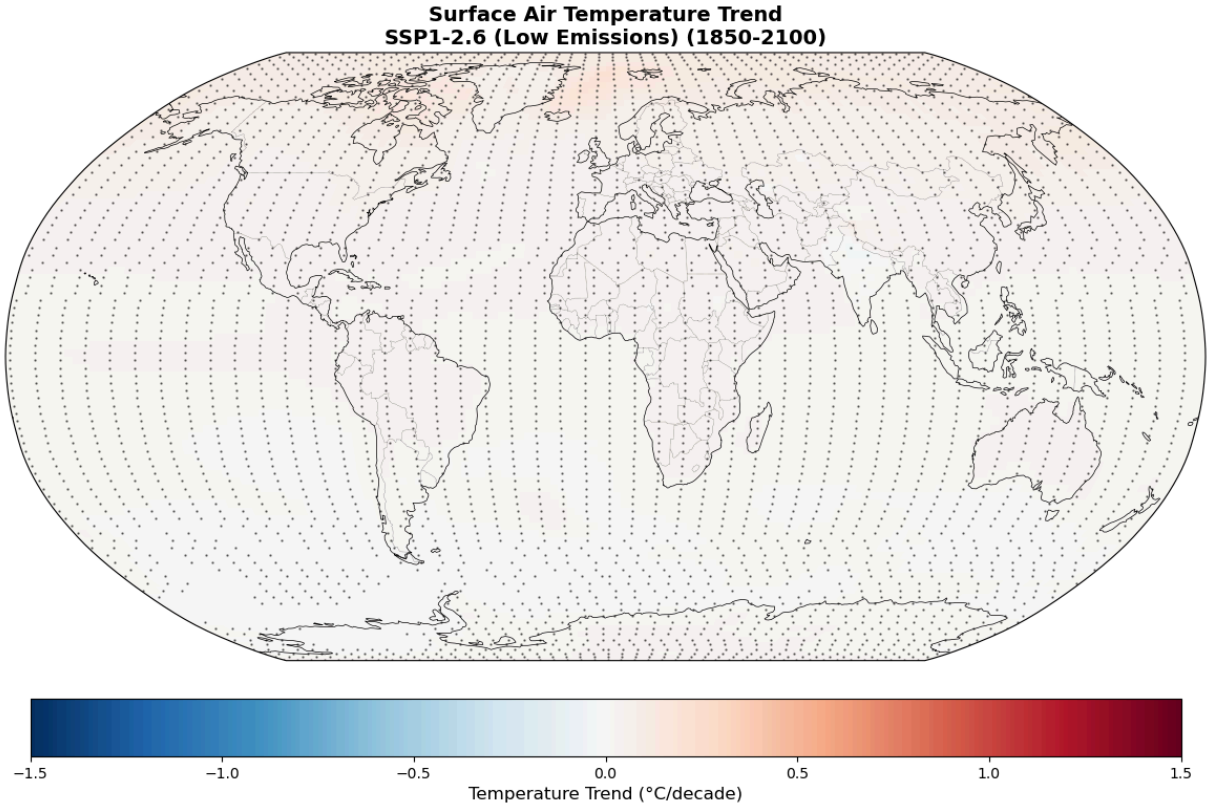

Figure 1: Surface Air Temperature Trends under SSP1-2.6

This map shows the spatial pattern of surface temperature trends from 1850-2100 under the low emissions scenario (SSP1-2.6). Red colors indicate warming trends while blue indicates cooling. The stippled areas show where trends are statistically significant ($p < 0.05$) across the majority of ensemble members. Under SSP1-2.6, we observe moderate warming across the globe, with enhanced warming over high-latitude land areas, particularly in the Arctic. This "Arctic amplification" occurs due to ice-albedo feedback mechanisms. The warming is less pronounced over oceans due to their higher heat capacity. The widespread stippling indicates high confidence in these projected changes.

In [13]:
```
# TAS SSP370
fig = plot_trend_map(
    ensemble_stats['ssp370']['tas']['mean_trend'],
    ensemble_stats['ssp370']['tas']['significance'],
    ensemble_stats['ssp370']['tas']['lat'],
    ensemble_stats['ssp370']['tas']['lon'],
    'tas', 'ssp370')
plt.show()
```

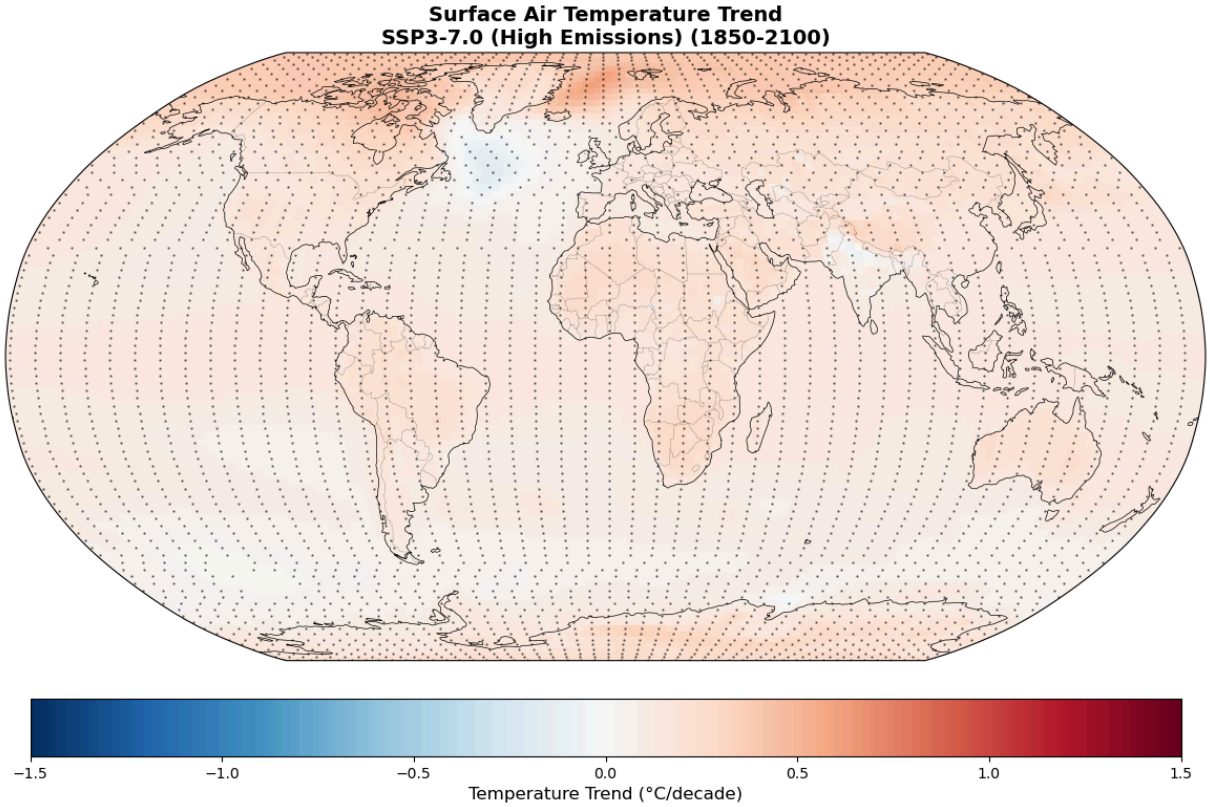

Figure 2: Surface Air Temperature Trends under SSP3-7.0

This map displays temperature trends under the high emissions scenario (SSP3-7.0). Compared to SSP1-2.6, the warming trends are substantially stronger across all regions. The Arctic shows the most dramatic amplification, with trends exceeding 1°C per decade in some areas. Continental interiors also show enhanced warming compared to coastal regions and oceans. The nearly complete coverage of stippling indicates that these warming trends are highly robust across all ensemble members. This scenario represents continued reliance on fossil fuels, resulting in much more severe warming impacts.

Precipitation (PR) Trend Maps

In [14]:
```
# PR SSP126
fig = plot_trend_map(
    ensemble_stats['ssp126']['pr']['mean_trend'],
    ensemble_stats['ssp126']['pr']['significance'],
    ensemble_stats['ssp126']['pr']['lat'],
    ensemble_stats['ssp126']['pr']['lon'],
    'pr', 'ssp126')
plt.show()
```

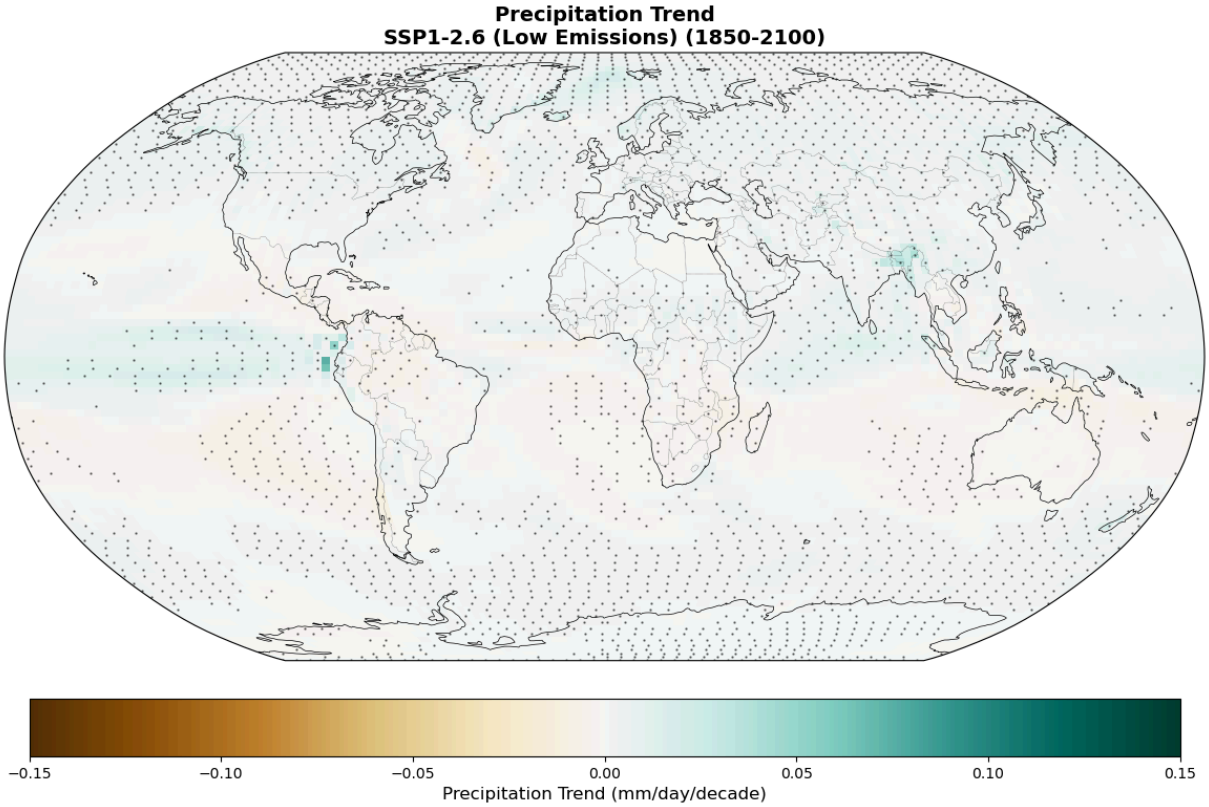

Figure 3: Precipitation Trends under SSP1-2.6

This map shows precipitation trends under the low emissions scenario. Green colors indicate increasing precipitation while brown indicates decreasing precipitation (drying). The pattern reveals the "wet-get-wetter, dry-get-drier" paradigm of climate change. Tropical regions show increasing precipitation due to enhanced atmospheric moisture in a warmer climate. High latitudes also show precipitation increases. Subtropical regions show drying trends associated with the expansion of the Hadley cell. The stippled areas indicate where these changes are statistically significant across ensemble members.

In [15]:
```
# PR SSP370
fig = plot_trend_map(
    ensemble_stats['ssp370']['pr']['mean_trend'],
    ensemble_stats['ssp370']['pr']['significance'],
    ensemble_stats['ssp370']['pr']['lat'],
    ensemble_stats['ssp370']['pr']['lon'],
    'pr', 'ssp370')
plt.show()
```

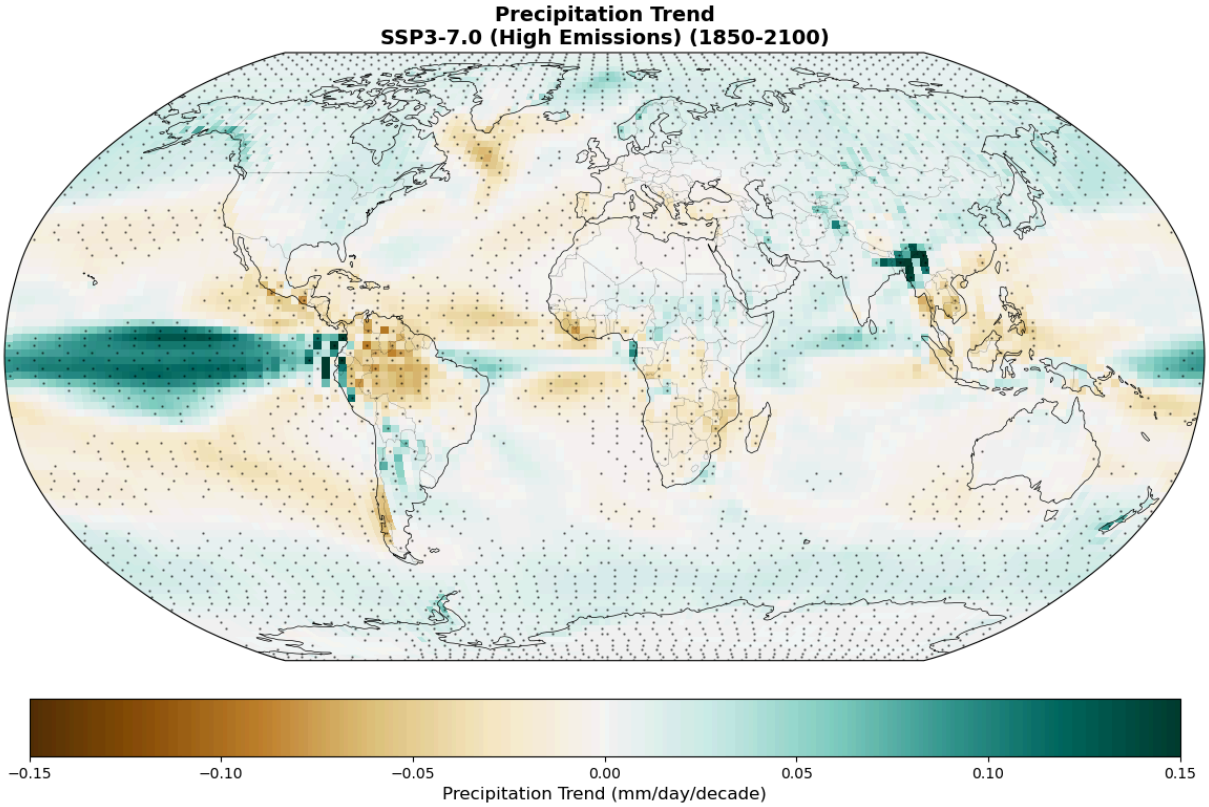

Figure 4: Precipitation Trends under SSP3-7.0

Under the high emissions scenario, precipitation changes are more pronounced. The contrast between wet and dry regions intensifies, with stronger increases in the tropics and high latitudes, and more severe drying in the subtropics. These changes have significant implications for water resources, agriculture, and ecosystem function. Regions already experiencing water stress may face more severe droughts, while flood risks may increase in wetter regions.

Soil Moisture (MRSOS) Trend Maps

In [16]:
```
# MRSOS SSP126
fig = plot_trend_map(
    ensemble_stats['ssp126']['mrsos']['mean_trend'],
    ensemble_stats['ssp126']['mrsos']['significance'],
    ensemble_stats['ssp126']['mrsos']['lat'],
    ensemble_stats['ssp126']['mrsos']['lon'],
    'mrsos', 'ssp126')
plt.show()
```

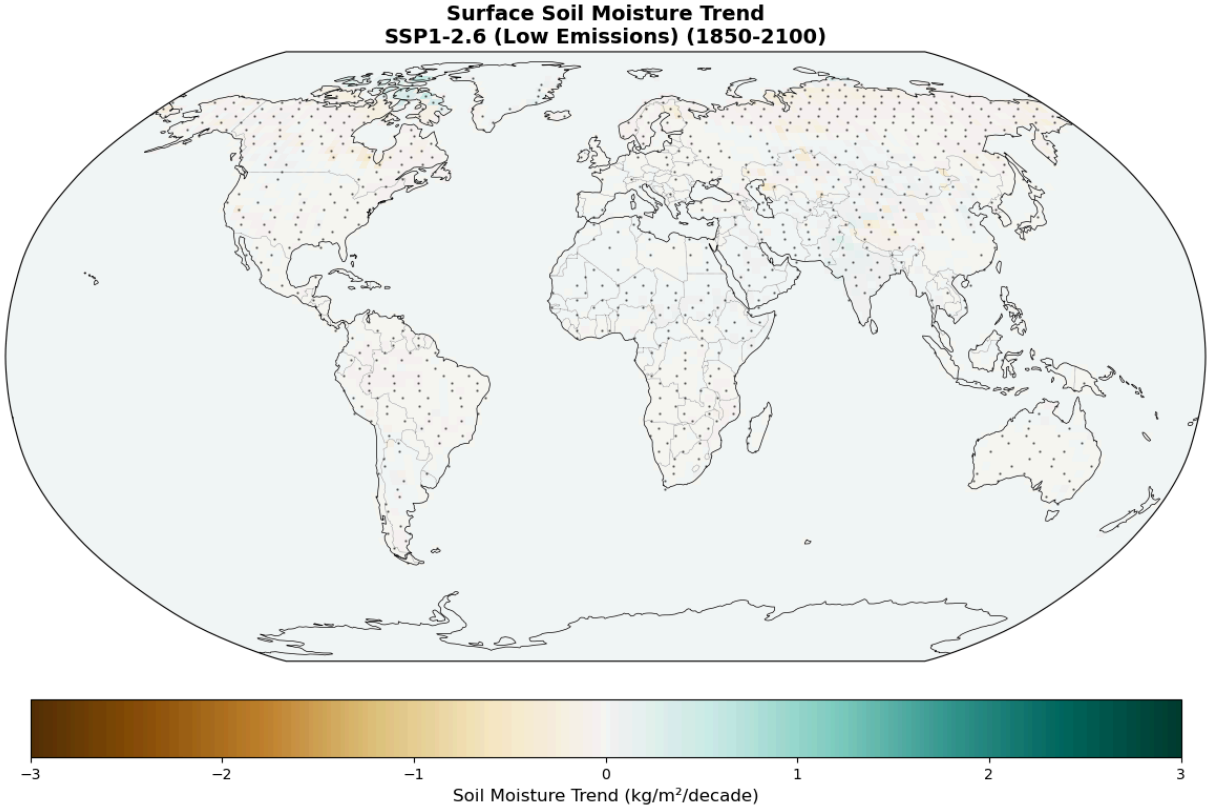

Figure 5: Surface Soil Moisture Trends under SSP1-2.6

This map shows trends in surface soil moisture under the low emissions scenario. Brown indicates drying soils while green shows increasing soil moisture. Soil moisture responds to the combined effects of precipitation changes and increased evapotranspiration from warming. Even in regions where precipitation increases, higher temperatures can lead to net soil moisture decreases due to enhanced evaporation. The Mediterranean region, Central America, and parts of southern Africa show notable drying trends. These changes have critical implications for agriculture and fire risk.

In [17]:
```
# MRSOS SSP370
fig = plot_trend_map(
    ensemble_stats['ssp370']['mrsos']['mean_trend'],
    ensemble_stats['ssp370']['mrsos']['significance'],
    ensemble_stats['ssp370']['mrsos']['lat'],
    ensemble_stats['ssp370']['mrsos']['lon'],
    'mrsos', 'ssp370')
plt.show()
```

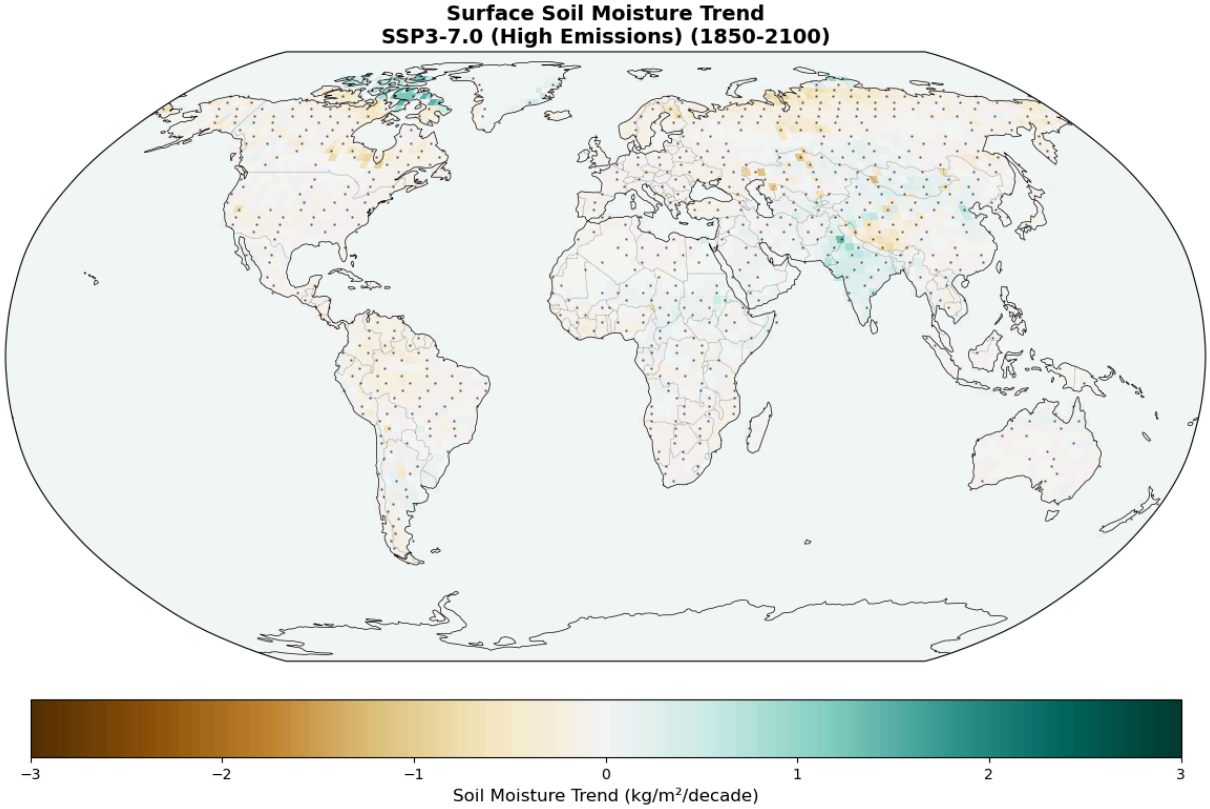

Figure 6: Surface Soil Moisture Trends under SSP3-7.0

Under the high emissions scenario, soil moisture changes are more severe. Widespread drying is projected over most continental areas, particularly in the subtropics and mid-latitudes. This occurs because higher temperatures drive increased evapotranspiration, which often outpaces precipitation increases. These changes would have profound impacts on agriculture, water availability, and wildfire risk. The contrast with SSP1-2.6 demonstrates the potential benefits of emissions mitigation for preserving soil water resources.

Part 10: Create Global Mean Time Series

Time series plots show how the global average of each variable evolves over time. We calculate area-weighted global means (accounting for the fact that grid cells are smaller near the poles) and plot:

- Individual ensemble members (showing internal variability)
- The ensemble mean (showing the forced response)
- Uncertainty range (±1 standard deviation)

In [18]:
```
def calculate_global_mean_timeseries(data_dict, variable):
    """Calculate area-weighted global mean time series."""
    time_series_list = []
    time_values = None
```

```
    for member, ds in data_dict.items():
        var_data = ds[variable]
        weights = np.cos(np.deg2rad(var_data.lat))
        weighted = var_data.weighted(weights)
        global_mean = weighted.mean(dim=['lat', 'lon'])
        time_series_list.append(global_mean.values)
        if time_values is None:
            time_values = var_data.time.values

    return time_values, time_series_list

def plot_timeseries(time, ts_list, variable, scenario):
    """Create time series plot with ensemble members and mean."""
    # Variable-specific parameters
    if variable == 'tas':
        ylabel = 'Temperature Anomaly (°C)'
        var_name = 'Surface Air Temperature'
    elif variable == 'pr':
        ylabel = 'Precipitation Anomaly (mm/day)'
        var_name = 'Precipitation'
        ts_list = [ts * 86400 for ts in ts_list]
    else:
        ylabel = 'Soil Moisture Anomaly (kg/m²)'
        var_name = 'Surface Soil Moisture'

    scenario_labels = {'ssp126': 'SSP1-2.6 (Low Emissions)', 'ssp370': 'SSP3-7.0 (H

    # Convert time to years
    years = np.array([int(str(t)[:4]) for t in time])

    fig, ax = plt.subplots(figsize=(12, 6))

    ts_array = np.array(ts_list)
    colors = ['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728']

    # Plot individual members
    for i, ts in enumerate(ts_list):
        ax.plot(years, ts, color=colors[i], alpha=0.4, linewidth=1, label=f'Member

    # Ensemble mean
    ensemble_mean = np.nanmean(ts_array, axis=0)
    ax.plot(years, ensemble_mean, color='black', linewidth=2.5, label='Ensemble Mea

    # Uncertainty shading
    ensemble_std = np.nanstd(ts_array, axis=0)
    ax.fill_between(years, ensemble_mean - ensemble_std, ensemble_mean + ensemble_s
                    alpha=0.2, color='gray', label='±1 Std Dev')

    ax.axhline(y=0, color='gray', linestyle='--', linewidth=0.8)
    ax.set_xlabel('Year', fontsize=12)
    ax.set_ylabel(ylabel, fontsize=12)
    ax.set_title(f'Global Mean {var_name} Anomaly\n{scenario_labels[scenario]} (Ref
                 fontsize=14, weight='bold')
    ax.legend(loc='upper left', fontsize=10)
    ax.grid(True, alpha=0.3)
```

```
    plt.tight_layout()
    return fig

print("Time series functions defined.")
```

```
Time series functions defined.
```

Temperature Time Series

In [19]:
```
# TAS SSP126
time, ts_list = calculate_global_mean_timeseries(anomaly_data['ssp126']['tas'], 'ta
fig = plot_timeseries(time, ts_list, 'tas', 'ssp126')
plt.show()
```

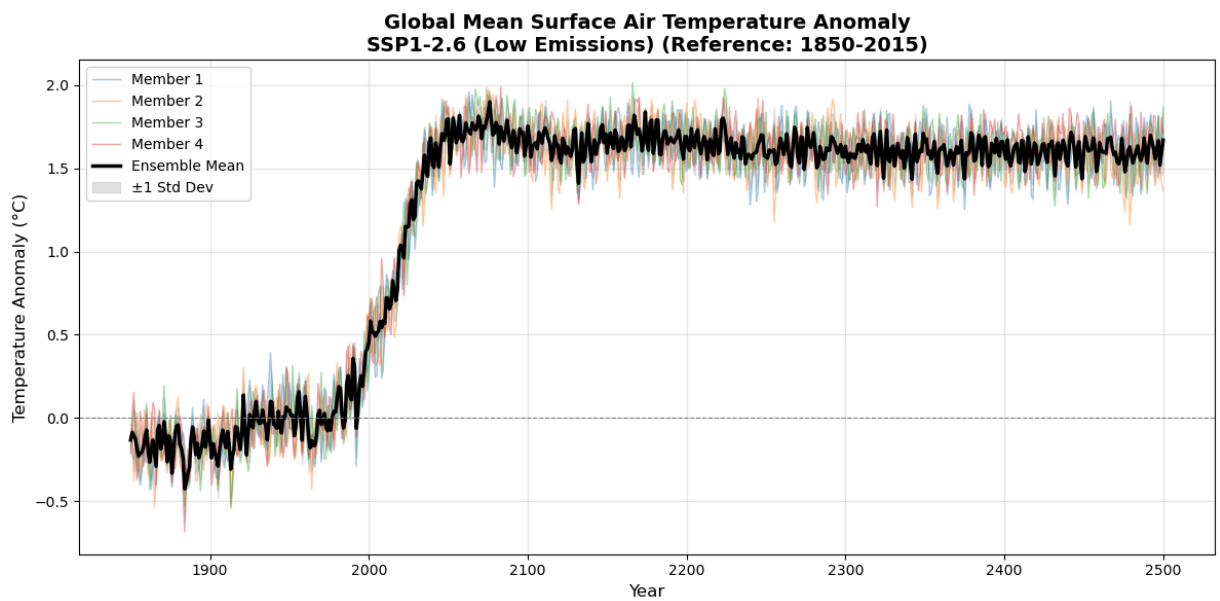

Figure 7: Global Mean Temperature Time Series - SSP1-2.6

This time series shows the evolution of global mean temperature anomalies from 1850 to 2100 under the low emissions scenario. The colored lines represent individual ensemble members, showing possible outcomes from different initial conditions. The black line shows the ensemble mean. Under SSP1-2.6, warming accelerates through the early 21st century before stabilizing as emissions decline. By 2100, global warming reaches approximately 1.5-2°C above the reference period.

In [20]:
```
# TAS SSP370
time, ts_list = calculate_global_mean_timeseries(anomaly_data['ssp370']['tas'], 'ta
fig = plot_timeseries(time, ts_list, 'tas', 'ssp370')
plt.show()
```

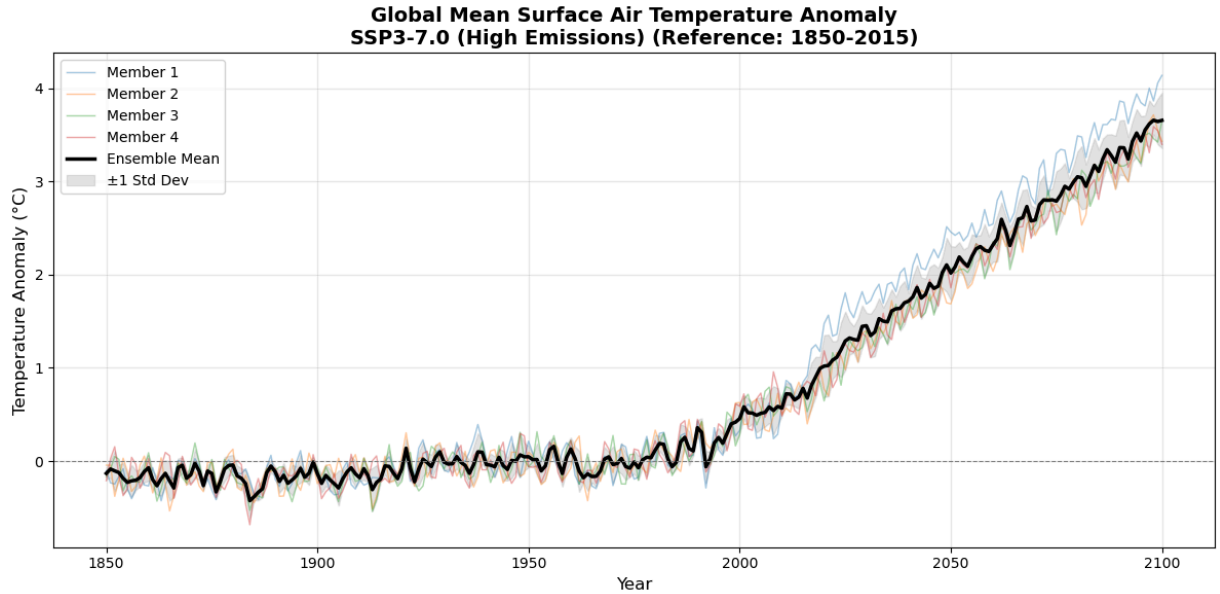

Figure 8: Global Mean Temperature Time Series - SSP3-7.0

Under the high emissions scenario, global temperature continues to rise throughout the century without stabilization. By 2100, global mean temperature anomalies exceed 3-4°C above the reference period. The contrast between this figure and Figure 7 demonstrates the critical importance of emissions reductions: the difference represents the potential impact of climate policy decisions made today.

Precipitation Time Series

In [21]:
```
# PR SSP126
time, ts_list = calculate_global_mean_timeseries(anomaly_data['ssp126']['pr'], 'pr'
fig = plot_timeseries(time, ts_list, 'pr', 'ssp126')
plt.show()
```

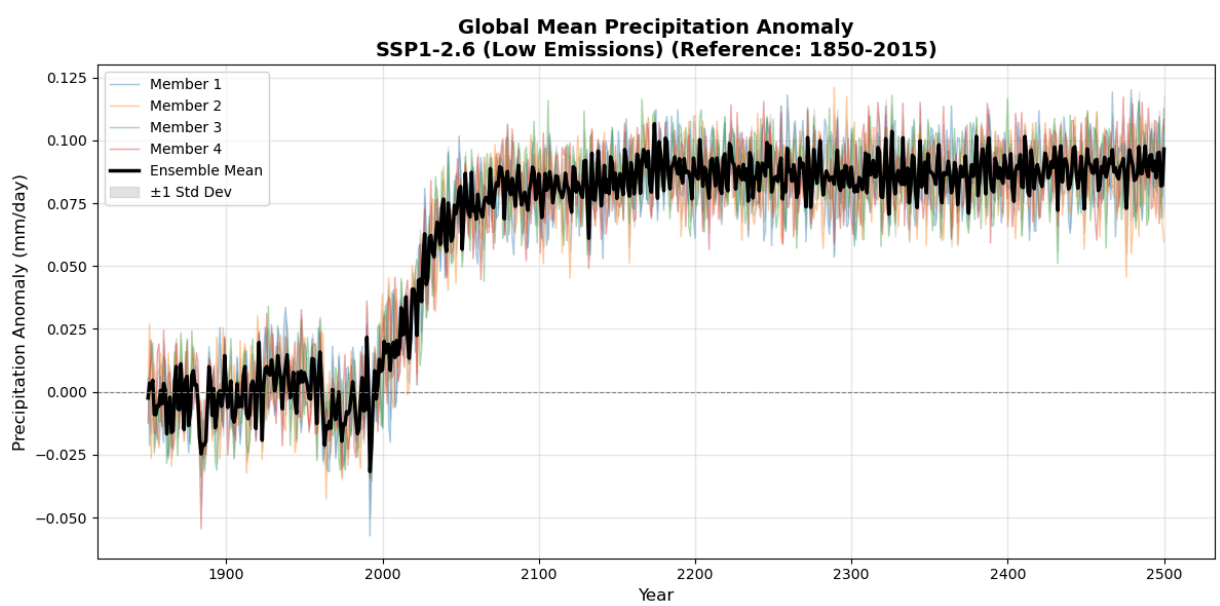

Figure 9: Global Mean Precipitation Time Series - SSP1-2.6

Global mean precipitation shows an increasing trend under SSP1-2.6, consistent with the Clausius-Clapeyron relationship: a warmer atmosphere holds more moisture, leading to an intensified hydrological cycle. The ensemble spread is larger than for temperature, reflecting greater internal variability in the water cycle. This global mean increase masks important regional variations.

In [22]:
```
# PR SSP370
time, ts_list = calculate_global_mean_timeseries(anomaly_data['ssp370']['pr'], 'pr'
fig = plot_timeseries(time, ts_list, 'pr', 'ssp370')
plt.show()
```

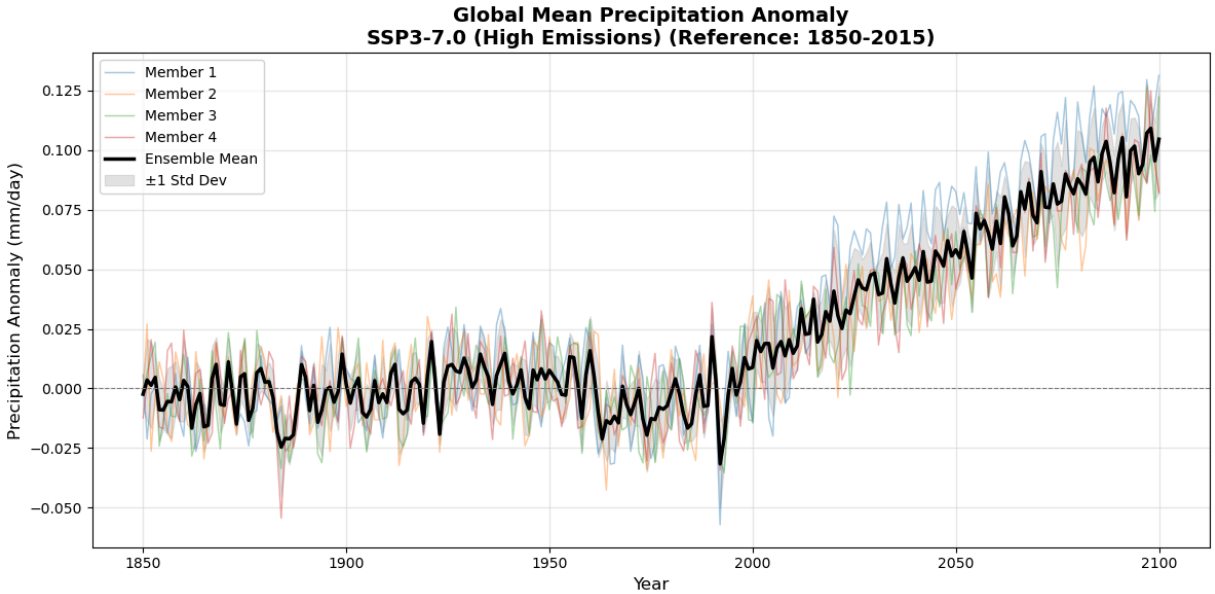

Figure 10: Global Mean Precipitation Time Series - SSP3-7.0

Under high emissions, global mean precipitation increases more substantially. The approximately 7% increase per degree of warming means more intense warming leads to greater hydrological intensification. These changes suggest a future with more extreme precipitation events globally.

Soil Moisture Time Series

In [23]:
```
# MRSOS SSP126
time, ts_list = calculate_global_mean_timeseries(anomaly_data['ssp126']['mrsos'], '
fig = plot_timeseries(time, ts_list, 'mrsos', 'ssp126')
plt.show()
```

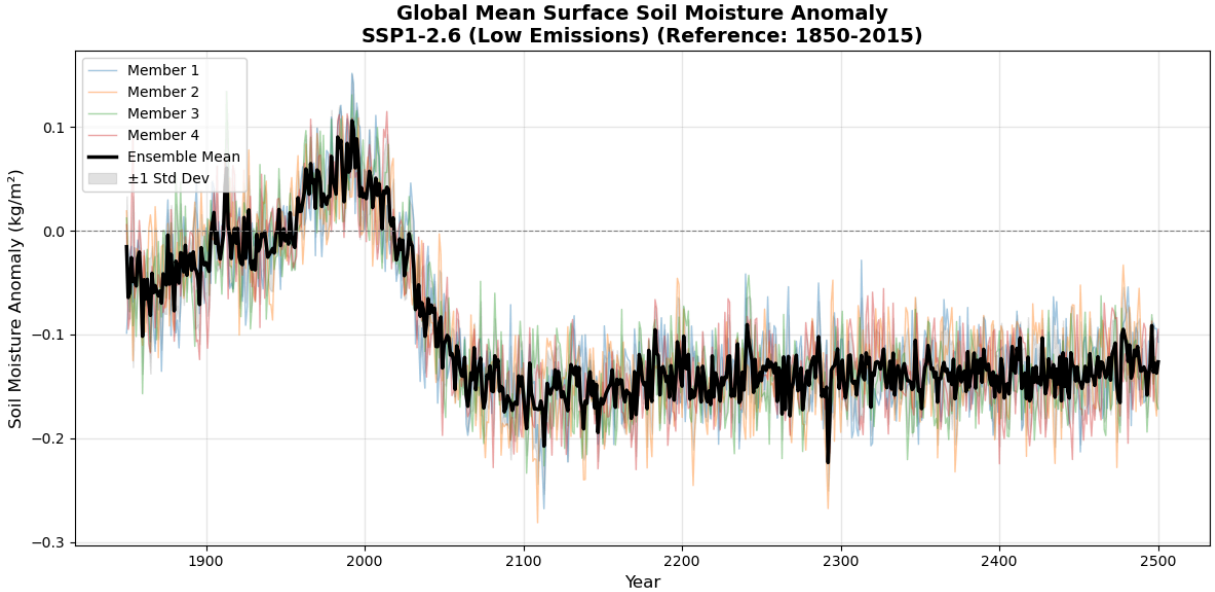

Figure 11: Global Mean Soil Moisture Time Series - SSP1-2.6

Global mean surface soil moisture shows a declining trend under SSP1-2.6, despite increasing global precipitation. This occurs because higher temperatures increase evapotranspiration rates, and over land areas, evaporative demand often exceeds precipitation increases. The modest decline under SSP1-2.6 suggests emissions mitigation could help preserve soil water resources.

In [24]:
```
# MRSOS SSP370
time, ts_list = calculate_global_mean_timeseries(anomaly_data['ssp370']['mrsos'], '
fig = plot_timeseries(time, ts_list, 'mrsos', 'ssp370')
plt.show()
```

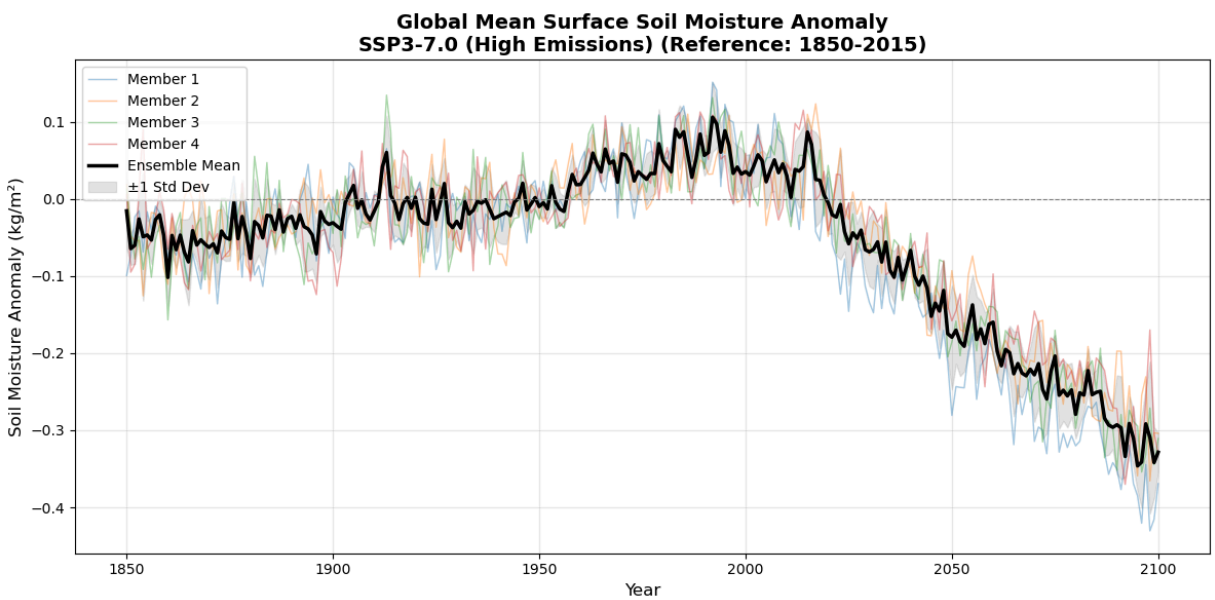

Figure 12: Global Mean Soil Moisture Time Series - SSP3-7.0

Under high emissions, global mean soil moisture declines more substantially throughout the century. Stronger warming drives increased evapotranspiration that more than offsets precipitation increases over land. This has profound implications for agriculture, water resources, and ecosystem health.

Part 11: Summary Statistics

In [25]:
```
print("=" * 70)
print("SUMMARY STATISTICS")
print("=" * 70)
print("\nGlobal Mean Trends (per decade):")
print("-" * 70)

for variable in variables:
    var_names = {'tas': 'Temperature', 'pr': 'Precipitation', 'mrsos': 'Soil Moistu
    units = {'tas': '°C/decade', 'pr': 'mm/day/decade', 'mrsos': 'kg/m²/decade'}

    print(f"\n{var_names[variable]} ({variable}):")

    for scenario in ['ssp126', 'ssp370']:
        mean_trend = np.nanmean(ensemble_stats[scenario][variable]['mean_trend'])
        if variable == 'pr':
            mean_trend *= 86400
        pct_sig = np.sum(ensemble_stats[scenario][variable]['significance']) / \
                  ensemble_stats[scenario][variable]['significance'].size * 100

        scenario_label = 'SSP1-2.6' if scenario == 'ssp126' else 'SSP3-7.0'
        print(f"  {scenario_label}: {mean_trend:.4f} {units[variable]} ({pct_sig:.1

print("\n" + "=" * 70)
print("ANALYSIS COMPLETE")
print("=" * 70)
```

```
======================================================================
SUMMARY STATISTICS
======================================================================

Global Mean Trends (per decade):
----------------------------------------------------------------------

Temperature (tas):
  SSP1-2.6: 0.0363 °C/decade (96.3% significant)
  SSP3-7.0: 0.1717 °C/decade (99.4% significant)

Precipitation (pr):
  SSP1-2.6: 0.0018 mm/day/decade (61.6% significant)
  SSP3-7.0: 0.0054 mm/day/decade (69.8% significant)

Soil Moisture (mrsos):
  SSP1-2.6: -0.0031 kg/m²/decade (19.7% significant)
  SSP3-7.0: -0.0095 kg/m²/decade (19.5% significant)

======================================================================
ANALYSIS COMPLETE
======================================================================
```

Conclusions

This analysis of CMIP6 climate model data reveals significant differences between the low emissions (SSP1-2.6) and high emissions (SSP3-7.0) scenarios:

Temperature:

- Both scenarios show global warming, but SSP3-7.0 produces warming roughly twice as severe
- Arctic amplification is evident, with high latitudes warming faster than the global average
- Temperature trends are statistically significant across nearly all grid points

Precipitation:

- Global mean precipitation increases due to atmospheric moistening
- Regional patterns follow the "wet-get-wetter, dry-get-drier" paradigm
- Changes are more pronounced under SSP3-7.0

Soil Moisture:

- Despite increasing precipitation, global soil moisture declines due to enhanced evapotranspiration
- Drying is more severe under SSP3-7.0
- Critical implications for agriculture and water resources

Key Takeaway: The comparison demonstrates that emissions pathway choices made today will have profound consequences for Earth's climate through 2100 and beyond.

Analysis performed using NASA-GISS GISS-E2-1-G model output from CMIP6.

In [ ]:

```

```